



Takneek PS - High Prep



Coding with agents has now become an integral part of every developer's workflow. The capabilities of frontier models continue to grow at an exponential rate, and their prices along with them. Hence many developers now turn to cheaper, open-weight models to provide the answer. However modern agentic coding systems, while featuring a plug-n-play approach for models, are not totally optimized for this approach. They assume large context windows and complex reasoning and planning capabilities within a single model. Small open-weight models simply cannot do this reliably on their own. They fall apart on multi-step reasoning tasks.

Hence it calls for systems that are designed around their limits from the ground up. This means using many small models working together while keeping costs under tight control.

Task / Problem Statement:

Build an **agentic coding IDE** that is built and tuned specifically for small and medium open-weight models (parameter count, total not active, $\leq 80B$), running only on **free-tier or pay-as-you-go APIs or your own local hardware**. The system must get the best possible accuracy on hard, multi-step coding tasks, while keeping token costs first and execution time second as low as possible.

1. Multi-Agent Orchestration:

- a. No single small model can carry a complex coding task on its own. It is only possible through a well thought out system of multiple agents working together.
- b. A **full system architecture** must be designed to carry a hard, multi-step coding task from the initial prompt to a completed, working result. This includes accounting for every stage a task may pass through, as well as the failure modes, disagreements, and setbacks that can occur along the way, and how the system is built to handle them.
- c. The system must detect when a task is stuck, for example, when repeating the same failed step, looping without making progress, or spawning agents endlessly and stop or step in instead of running forever. You are free to define additional safeguards of



Takneek PS - High Prep



your own, for instance, limits on retry attempts, total steps taken, or tokens consumed within a task.

2. Model and Hosting Constraints:

- a. Every model used anywhere in the system must have a **total parameter count** (not active/expert count) of **80B or less**.
- b. **Only free-tier or pay-as-you-go provider APIs** or models running on **local hardware** are allowed anywhere in the pipeline. Usage of **paid subscription-based APIs** of any kind are **not allowed**.
- c. The models you chose to run on local hardware, must comfortably run on a device with **16GB RAM** and **8GB VRAM**.

3. Smart Routing:

- a. For every prompt, the system must automatically decide which model and which provider should handle it. This decision must be based on real, sensible signals such as
 - Task complexity
 - size of context needed
 - Tokens already used
 - Provider rate limits
 - Any other rule you design and can justify.
- b. **The routing decision can never be hidden.** The user must always be able to see live, exactly which model and provider is handling which piece of work, and ideally why.
- c. If a model call fails or a rate limit is hit, the system must recover gracefully by falling back to another model or provider, **without losing the task's progress**.
- d. Implement a simple settings screen to let the user type in and store their own API keys for each provider you support.

IMPORTANT!

This settings screen is **mandatory**. Teams that do not implement it will be **disqualified**, since we need it to test your system during evaluation.

4. Automatic Context Compaction:

- a. The system must, on its own, decide the best time to compress the running context of a task, and must recognise when compaction is absolutely necessary to avoid crashing into a context limit.



Takneek PS - High Prep



- b. After compaction, information from earlier in the task that is still relevant must not be lost or misremembered.
- 5. **Code Retrieval Pipeline:**
 - a. You must design an efficient codebase index, along with a search and selection strategy for retrieval, together forming a high-performing code retrieval pipeline.
 - b. A codebase is the root folder the user opens in the IDE. Each codebase gets its own separate index. **Retrieval and agent memory from one project must never leak into another project.**
- 6. **Long-Horizon, Multi-Session Tasks:**
 - a. **A complex task can be much longer than any single context window**, and can also be interrupted, maybe the user closes the IDE, a session times out, or a crash happens. Your architecture must be able to store the state and history of such a task, and later retrieve it to resume exactly where it left off, instead of starting the task over from scratch.
- 7. **Manual Context Control:**
 - a. Users must be able to manually add or remove files and selected code blocks from the active context at any time.
 - b. Both the input box and the output chat must support **clickable tagging of files** and specific code lines, so the user can point the agent to exact code and vice-versa.
 - c. A special **/bytheway** command must let the user ask one, **isolated question with zero prior context in the same chat window**, get an answer, and then return seamlessly to the original ongoing context.
- 8. **Autonomous Tool Use:**
 - a. **The agent must be able to run terminal commands**, request and read files, search the web, and carry out Git operations (branch, commit, diff, merge, etc.) entirely on its own, whenever the task calls for it.
 - b. Any command with a side effect i.e writing or deleting a file, pushing to Git, installing a package, or running a terminal command that changes state, must always require human approval before it runs.
- 9. **Style and Project Memory:**



Takneek PS - High Prep



- a. The system must track and apply per-project rules and preferences using the AGENTS.md protocol. It includes things like coding style, build and test commands, folder conventions, and any other project-specific instruction the file defines.

10. Human-in-the-Loop (HITL) Review:

- a. **The workflow must generate proper Git diffs for every change the agent proposes.**
- b. The user must be able to accept or reject changes block by block, not only as an all-or-nothing choice, along with simple "accept all" and "reject all" options.
- c. When the user only partially approves a set of changes, the agent must be able to continue executing the rest of the task correctly, working around the rejected parts.

11. Observability Dashboard:

- a. The IDE must include a **full tracing dashboard** for the current project.
- b. For every task, it must
 - i. Show the **complete call hierarchy** of every agent and tool involved
 - ii. Let the user drill into any single node to see its **exact input and output**,
 - iii. Show **live or logged thought process** of agents.
 - iv. Show the exact files or code chunks in each agent's context over their lifetime
 - v. The **tokens and time** each agent used.
- c. The user must be able to inspect this both while a task is running and after it has finished.

Deliverables:

Since this problem statement asks for a **full agentic system** and not a single feature, the deliverables go beyond just working code. Each of the following is expected:



Takneek PS - High Prep



1. **A .zip file with the working source code with full Git history:** The .git folder must be included so the actual development process can be checked.
2. **Cross-platform builds** for Windows, macOS, and Linux, along with clear steps to run each one on a clean machine.
3. **Documentation:**
 - a. The **project architecture and setup instructions (only for Linux)** from scratch, including **API key setup for each provider**.
 - b. The entire **multi-agent orchestration pipeline** explained along **with diagrams**.
 - c. Other architectural decisions, such as the tool-calling format used, along with the reasoning behind them.
4. **Presentation**
 - a. Which you will be presenting later.
 - b. Presentation time will be a maximum of 10 minutes.
 - c. At least **2 members** will be presenting.

Deadline: 2359 hrs, 2nd September 2026

Evaluation Metrics:

For each weighted component below, we've listed some of the questions we'll be asking ourselves while evaluating your submission. This list is not exhaustive, it's meant to give you a sense of what to think about, not a complete checklist..

1. **Final System Performance, end to end (20%)**
 - a. Measured on a fixed hidden set of codebases and tasks the same for every team.
 - b. Scored on a mix of accuracy of the final code change, total token cost (in dollars), and total time taken, combined into one score.
 - c. **The final scoring formula will be as follows :**

$$S_{task} = \frac{10 \cdot A}{\left(1 + w_c \left(\frac{C}{C_{base}}\right) + w_T \left(\frac{T}{T_{base}}\right)\right)^\epsilon}$$

- d. Where the notations are explained below:



Takneek PS - High Prep



- i. **Accuracy** ($A \in [0, 1]$): Computed as $\frac{PassedTests}{TotalTests}$ on our evaluation tasks
 - ii. **Cost** (C): Total dollar cost incurred across all agent calls
 - iii. **Execution Time** (T): Total wall-clock time in seconds from prompt input to final output generation.
 - iv. **Penalty Exponent** (ϵ): Set to **2.5** wherein you will be heavily penalized if the token cost exceeds the maximum cost allowed as per the constraints
 - v. **Reference Constraints:**
 - $C_{base} = \$0.15$
 - $T_{base} = 1320$ seconds
 - $w_T = 0.35$ and $w_C = 0.65$.
 - Note that the base time and costs given are not targets. Just baseline reference costs
 - e. Each evaluation task is also subject to a **maximum time** of **2700 seconds** and a **budget of \$0.5**. Any task that exceeds this budget will be halted and scored as a failure ($A = 0$) for that task, regardless of partial progress. This is a hard ceiling for evaluation purposes and is separate from the cost penalty in the scoring formula, which applies below this limit.
 - f. The hidden evaluation set consists of medium-to-hard, long-horizon coding tasks drawn from real-world codebases, requiring multi-step reasoning to solve, not single-shot fixes
2. **Core Agentic Architecture (36%)**
- a. **Routing (5%)**
 - i. Does it correctly send easy tasks to smaller or cheaper models and hard tasks to bigger ones?
 - ii. How smooth is the fallback for API error/limit? Is task progress preserved, or does a failure cause visible disruption or repeated work?
 - iii. How transparent is the routing decision?
 - b. **Compaction (5%)**
 - i. How frequently does compaction occur?



Takneek PS - High Prep



- ii. How well does the system work with multiple compaction events in the same task?
 - c. **Code Retrieval (12%)**
 - i. Does it avoid returning irrelevant or excess code, with no dumping of whole files?
 - ii. Does the codebase index understand the code semantically, its structure, logic and execution flow?
 - iii. How well does it work across multiple files?
 - iv. How does the system detect and recover when an initial retrieval attempt returns poor or incomplete results?
 - v. Does the retrieval approach go meaningfully beyond simple keyword matching or plain vector embeddings, and is that choice well justified?
 - d. **Orchestration (14%)**
 - i. How well does it break a big task into smaller, well-scoped sub-tasks?
 - ii. How efficiently is the overall plan carried out
 - iii. Is each step given only the context it actually needs, and nothing more?
 - iv. When something fails, does the system properly diagnose and fix it, instead of blindly retrying the same action?
 - v. Can it backtrack, undo a bad step or direction and re-plan, instead of getting stuck or piling more errors on top of old ones?
 - vi. Does it review or verify its own output before calling a task as done?
 - vii. How well does the system avoid repeating work or letting parallel agents make conflicting changes?
 - viii. How well does the failsafe against stuck or runaway tasks work?
- 3. **Supporting Agentic Features (17%)**
 - a. **Manual context control (6%)**
 - i. Can files and code blocks be added or removed from context easily?



Takneek PS - High Prep



- ii. Do clickable file and line tags work correctly in both the input box and the output chat?
 - iii. Does the /bytheway command correctly isolate one prompt and return cleanly to the original context?
 - b. **Human-in-the-loop review (3%)**
 - i. Are the generated Git diffs accurate and easy to read?
 - ii. Does block-level accept/reject work correctly?
 - iii. Is partial approval handled correctly, without breaking the rest of the task?
 - c. **Terminal, file, web, and Git autonomy (6%)**
 - i. Do agents actively use these tools as feedback mechanisms (to verify results) and to log progress?
 - ii. Does it ask for human approval before running any command with a side effect?
 - d. **AGENTS.md style and preference handling (2%)**
 - i. Is a stated preference or rule actually respected in later output?
 - ii. Does this hold even after a compaction event?
- 4. **Dashboard, User Interface and Experience (12%)**
 - a. **Observability dashboard (8%)**
 - i. How complete and accurate is the call hierarchy and per-node detail?
 - ii. Is the dashboard equally usable while a task is running and after it has finished, with no gaps between the two modes?
 - b. **General IDE usability (4%)**
 - i. Is the day to day experience of writing code, reviewing diffs, and managing context actually pleasant?
 - ii. Is the design minimal, functional and easy to use?
- 5. **Documentation, Code Quality and Presentation (15%)**
 - a. **Documentation (7%)**
 - i. **System architecture:** How well do you explain the actual architecture of your agentic system?
 - ii. **Trade-off analysis:**



Takneek PS - High Prep



1. Did you explain why you made key architectural decisions, and why you rejected other approaches?
2. Did you actually try and compare more than one approach for any core component, instead of going with the first idea that worked?
- iii. **Setup clarity:** Are the setup steps clear enough for someone who has never seen the codebase to get it running from scratch on Linux, including API key setup for every provider used?
- iv. **Challenges and solutions:** What real problems came up while building the system, and how were they solved?
- b. **Code quality (2%)**
 - i. Is the code readable and modular, with meaningful names and minimal duplication?
- c. **Presentation (6%)**
 - i. The Q&A during your presentation is used to adjust scores across all sections in this document.

Note:

- The weightage next to each component is not used for objective scoring, it only gives participants a sense of relative importance.
- All scoring, including these weightages, is subjective, judgement based, and subject to change.
- For the architectural components in particular (routing, compaction, retrieval, orchestration), judges will favor solutions that show original thinking, well-reasoned trade-offs, and a deep understanding of the problem over solutions that simply follow the most common or first-suggested approach.
- Two systems that "work" equally well on the surface may score differently if one handles edge cases more smartly, justifies its method choices, or tries something non-obvious that improves performance.



Takneek PS - High Prep



Guidelines:

Strict alterations in evaluation procedures will be taken in order to judge the problem statement on the merits of the developers and not the merits of the Large Language Model or the Vibe Coding platform. These shall, again be highly subject to the experience of the evaluator, but we shall try to list down the same in a structured fashion:

1. **Focus on knowledge during presentation:** If developers are not able to answer any question on a feature or component during the presentation, but the same is present in the codebase, the implementation of the feature shall be considered null and void.
 2. **Focus on functionality over aesthetics:** While a visually appealing platform is important, functionality comes first. An intuitive, easy-to-use experience will always be prioritized over flashy design.
 3. **Clear description and understanding of system architectures and tradeoff considerations:** Large Language Models often come up with standard procedures, use standard tools, program functionalities in a certain fashion. Think beyond the suggestion of the LLM, research about various optimizations, understand tradeoffs and do some research from ground up. Read blogs, understand why certain tools are preferred over the others in certain scenarios etc. These explanations should also be clearly mentioned in the README, and the participants should have a detailed understanding of these explanations. If we find that the explanations are AI-generated and participants are not able to answer questions, heavy penalties will be applied.
 4. Lastly, even if the quality of one solution is objectively better than the other - we might choose to judge the objectively worse solution as the better one in case the developers of the prior are not able to explain their developments.
5. **On involvement of Y26s:** In past competitions, we have observed a recurring issue where freshers do not actively participate in problem solving and are merely coached on specific sections for the presentation. To prevent this deceptive practice, presentations will now feature two distinct Q&A sessions: a general session for the entire team, and a dedicated session exclusively for Y26 students. The Y26 Q&A will go beyond generic concepts to test specific, implementation-based knowledge. If it becomes evident that the



Takneek PS - High Prep



freshers did not actively contribute to building the project from scratch, the team will face severe penalties. We shall not reveal more on the exact pattern of the Y26 Q&A but broadly they will be tested on their fundamentals, contribution and participation in the PS.

Team Composition :

1. **Team Composition:** Each team should have **8 members**, with at least **3 Y26s** and **3 Y25s**.

For Any Queries, The Pool Captains and PS Leads are encouraged to ask in the Discord channel.

